

COMPANION
to

ChatGPT Teaches TikZ

Chapter 6: Positioning (B)

Contents

6.1	What This Companion Adds	2
6.2	A Positioning Vocabulary	3
6.3	Reading a Placement Option	3
6.4	Rows and Columns	4
6.5	Positioning Around One Node	5
6.6	Controlling Distance	6
6.7	Worked Project: A Branching Process	6
6.8	Debugging Workshop	9
6.9	Exercises	10
6.10	Solutions	10
6.11	ChatGPT Workshop	12
6.12	Final Checklist	12

6.1 What This Companion Adds

Chapter 6 replaced unrelated coordinates with relationships among named nodes. This Companion develops that idea into a reliable method for rows, columns, branches, and flowcharts.

The central question is no longer “Where is this node?” It is “How is this node related to the one already placed?” That change makes a diagram easier to read, revise, and extend.

Design

Use relative positioning when relationships define the layout. Keep raw coordinates for objects whose measured positions are themselves important.

6.2 A Positioning Vocabulary

Form	Purpose
<code>positioning</code>	Loads the positioning library.
<code>right=of A</code>	Places a node to the right of node A.
<code>left=of A</code>	Places a node to the left of A.
<code>above=of A</code>	Places a node above A.
<code>below=of A</code>	Places a node below A.
<code>below right=of A</code>	Places a node diagonally from A.
<code>node distance=15mm</code>	Sets the normal separation.
<code>right=25mm of A</code>	Overrides the normal separation once.
<code>on grid</code>	Uses anchor-to-anchor positioning.

Load the library in the preamble:

```
\usetikzlibrary{positioning}
```

6.3 Reading a Placement Option

Consider

```
\node[draw,right=of input] (process) {Process};
```

The visible word is **Process**, the new internal name is **process**, and the reference node is **input**. TikZ places the new node only after it finds the already defined node named **input**.

Common Mistake

The phrase **right=of Input** is wrong if **Input** is only visible text. A positioning option must use the node's internal name.

6.4 Rows and Columns

One shared distance can control an entire row.

```
\begin{tikzpicture}[node distance=14mm]
  \node[draw] (A) {Input};
  \node[draw,right=of A] (B) {Process};
  \node[draw,right=of B] (C) {Output};
\end{tikzpicture}
```

See

Input

Process

Output

The same method builds a column: replace **right=of** with **below=of**. If the first node moves, every later node still follows the chain of relationships.

6.4.1 Why the Order Matters

The reference node must exist before it is used. This order is correct:

```
\node[draw] (first) {First};
\node[draw,below=of first] (second) {Second};
```

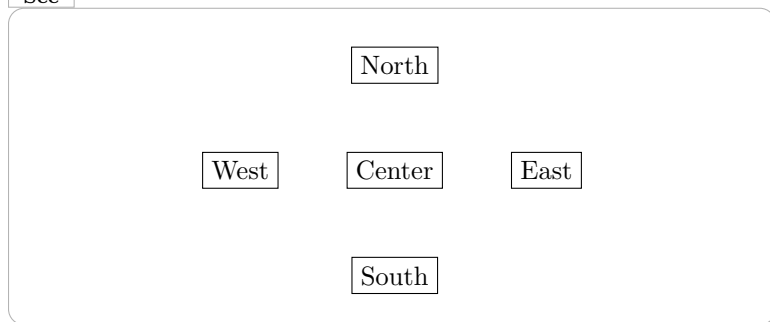
Reversing the two commands asks TikZ to position a node relative to a name it has not yet encountered.

6.5 Positioning Around One Node

The four directions and their diagonal combinations can describe a small neighborhood without any numerical coordinates.

```
\begin{tikzpicture}[node distance=12mm]
\node[draw] (center) {Center};
\node[draw,above=of center] (north) {North};
\node[draw,below=of center] (south) {South};
\node[draw,left=of center] (west) {West};
\node[draw,right=of center] (east) {East};
\end{tikzpicture}
```

See



For a branch, use a diagonal direction such as `below left=of center` or `below right=of center`.

6.6 Controlling Distance

Set the ordinary spacing once:

```
\begin{tikzpicture}[node distance=16mm]
...
\end{tikzpicture}
```

Then use an explicit distance only for a genuine exception:

```
\node[draw,right=28mm of A] (B) {A longer label};
```

This makes the exception visible in the source. A collection of small shifts usually hides the intended structure.

6.6.1 What on grid Changes

Ordinary positioning separates node borders. With `on grid`, TikZ uses the relevant anchors, so unequal node sizes do not change the center-to-center spacing.

```
\begin{tikzpicture}[node distance=25mm,on grid]
  \node[draw] (A) {Short};
  \node[draw,right=of A] (B) {A much longer node};
\end{tikzpicture}
```

Use `on grid` when a regular alignment matters more than equal gaps between borders.

6.7 Worked Project: A Branching Process

The project begins with relative placement and adds connections only after the layout is complete.

6.7.1 Stage 1: Define a Shared Style

```
\tikzset{
  process node/.style={draw,rounded corners}
}
```

6.7.2 Stage 2: Place the Main Column

```
\node[process node] (start) {Start};
\node[process node,below=of start] (read) {Read data};
\node[process node,below=of read] (check) {Valid?};
```

6.7.3 Stage 3: Add Two Branches

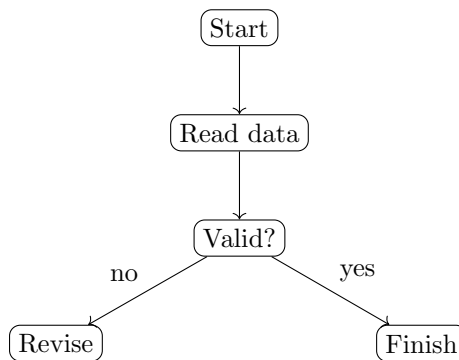
```
\node[process node,below left=of check] (revise) {Revise};
\node[process node,below right=of check] (finish) {Finish};
```

6.7.4 The Complete Picture

```
\begin{tikzpicture}[
  node distance=11mm and 14mm,
  process node/.style={draw,rounded corners}
]
  \node[process node] (start) {Start};
  \node[process node,below=of start] (read) {Read data};
  \node[process node,below=of read] (check) {Valid?};
  \node[process node,below left=of check] (revise) {Revise};
  \node[process node,below right=of check] (finish) {Finish};

  \draw[->] (start) -- (read);
  \draw[->] (read) -- (check);
  \draw[->] (check) -- node[above left] {no} (revise);
  \draw[->] (check) -- node[above right] {yes} (finish);
\end{tikzpicture}
```

See



Design

The source first records the placement relationships and then the logical connections. Changing the spacing does not require rewriting either branch.

6.8 Debugging Workshop

6.8.1 Forgetting of

Incorrect:

```
\node[draw,right=A] (B) {B};
```

Correct:

```
\node[draw,right=of A] (B) {B};
```

6.8.2 Using a Visible Label as a Name

If the node is declared as

```
\node[draw] (source) {Input data};
```

refer to `source`, not `Input data`.

6.8.3 Using a Name Before Defining It

Read positioned nodes from the reference outward. Every name appearing after `of` must already have been defined.

6.8.4 Repairing Every Gap Separately

If several gaps are wrong, change `node distance`. Use an explicit distance only when one relationship is intentionally different.

6.9 Exercises

1. Place four named nodes in a row using `right=of`.
2. Place three nodes in a column using `below=of`.
3. Put one node above and one node below a central node.
4. Add nodes diagonally below left and below right of a central node.
5. Change an entire row's spacing by editing one value.
6. Give one unusually long node an explicit distance.
7. Rebuild a coordinate-based three-node row with relative positioning.
8. Add a third outcome to the worked project's decision node.

6.10 Solutions

6.10.1 Exercises 1–4

```
\node[draw] (A) {A};  
\node[draw,right=of A] (B) {B};  
\node[draw,right=of B] (C) {C};  
\node[draw,right=of C] (D) {D};
```

```
\node[draw] (A) {A};  
\node[draw,below=of A] (B) {B};  
\node[draw,below=of B] (C) {C};
```

```
\node[draw] (C) {Center};
```

```
\node[draw,above=of C] (A) {Above};
\node[draw,below=of C] (B) {Below};
```

```
\node[draw] (C) {Center};
\node[draw,below left=of C] (L) {Left};
\node[draw,below right=of C] (R) {Right};
```

6.10.2 Exercises 5–7

```
\begin{tikzpicture}[node distance=20mm]
  \node[draw] (A) {A};
  \node[draw,right=of A] (B) {B};
  \node[draw,right=of B] (C) {C};
\end{tikzpicture}
```

```
\node[draw,right=30mm of A] (B) {A longer label};
```

```
\node[draw] (input) {Input};
\node[draw,right=of input] (work) {Work};
\node[draw,right=of work] (output) {Output};
```

For Exercise 8, add another positioned node and one arrow. Preserve the five existing placement relationships.

6.11 ChatGPT Workshop

Ask ChatGPT to preserve the logical arrangement unless a prompt explicitly requests a structural change.

1. Rewrite this coordinate-heavy node layout with the positioning library. Do not change the logical arrangement.
2. Explain every part of **below right=18mm of check**.
3. Adjust this flowchart by changing **node distance**, not by adding manual shifts.
4. Find every reference to an undefined node name in this TikZ picture.
5. Decide whether ordinary positioning or **on grid** better matches this layout, and explain the choice.

6.12 Final Checklist

Before continuing to Chapter 7, check that you can

1. load and use the positioning library;
2. distinguish a node's internal name from its visible text;
3. build rows, columns, and diagonal branches;
4. control ordinary spacing with **node distance**;
5. use an explicit distance for a genuine exception;
6. define each reference node before using it; and
7. revise a relative layout without recalculating coordinates.